AWS MUSIC STORAGE ASSIGNMENT

Name: Meet Kalal

Subject: Amazon Web Services (AWS)

Topic: S3 Bucket Security and Encryption

Q.1. Create a storage bucket named 'my-music-uploads' and upload any two MP3 files to it.

Answer: I will sign in to the AWS Management Console and open Amazon S3. I will select Create bucket and enter the bucket name **my-music-uploads**. I will choose my required AWS Region and keep Block Public Access enabled. After creating the bucket, I will open it, select Upload, and add two MP3 files from my computer, for example **song-one.mp3** and **song-two.mp3**. I will select Upload and check that both files appear in the Objects list.

Q.2. Apply a bucket policy so only a user with a specific email address can download files from the bucket.

Answer: In AWS, a bucket policy normally identifies an IAM user or role by its AWS ARN, not directly by an email address. I will first create or identify an IAM user for the intended person and note that user's ARN. I will then open S3 → my-music-uploads → Permissions → Bucket policy and apply a policy that allows the identified IAM user to list the bucket and download its objects. I will replace the example ARN below with the real IAM user ARN.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSpecificUserToListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/MusicUser"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::my-music-uploads"
    },
    {
      "Sid": "AllowSpecificUserToDownloadObjects",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/MusicUser"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-music-uploads/*"
    }
  ]
}
```

Note: The account ID and IAM user name above are examples. Replace them with the correct values. Also ensure no other bucket or identity policy grants access to other users. The selected user's IAM permissions and any organization-level controls must also permit the request.

Q.3. Enable server-side encryption for all objects uploaded to the bucket.

Answer: I will open the bucket's Properties tab and find Default encryption. I will select Edit, choose Server-side encryption with Amazon S3 managed keys (SSE-S3), and save the changes. After this setting is enabled, new objects uploaded to the bucket will be encrypted automatically by Amazon S3. I can verify this by opening an uploaded object's Properties and checking its Server-side encryption details.

Q.4. Test the bucket security by trying to access an MP3 file from a browser where I am not logged in.

Answer: I will copy the Object URL of one uploaded MP3 file and open it in a private or incognito browser window where I am not signed in to AWS. Since the bucket is private and access is restricted to the selected IAM user, the file should not be available to an anonymous visitor. The browser should show an Access Denied or AccessDenied error. I will take a screenshot of the actual result and attach it with my assignment.

Screenshot evidence: Paste the screenshot of the actual Access Denied result here after performing the test.

Q.5. Generate a sample bucket policy that restricts access to the bucket by IP address, then modify and apply it.

Answer: I used an AI tool to prepare the following sample AWS S3 bucket policy. It denies requests to the bucket and its objects when the request comes from an IP address outside the specified public IP range. I will replace the example IP address with my own or my college network's public IP address, review the policy, and apply it under Permissions → Bucket policy.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyRequestsOutsideApprovedIP",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::my-music-uploads",
        "arn:aws:s3:::my-music-uploads/*"
      ],
      "Condition": {
        "NotIpAddress": {
          "aws:SourceIp": "203.0.113.10/32"
        }
      }
    }
  ]
}
```

Important: 203.0.113.10/32 is a documentation-only example IP and must be replaced with the real public IP address before applying the policy. This explicit Deny can block even the intended IAM user when they connect from another network. Keep the AWS Console session open, confirm the correct IP first, and test carefully so you do not lock yourself out. If using a college network, its public IP may be shared or may change.

Conclusion: Amazon S3 can store music files securely using IAM-based access control, private bucket settings, server-side encryption, and IP-based restrictions. I will verify the upload, encryption status, and denied anonymous access in my AWS account and attach real screenshots as proof.

Practical evidence to attach: (1) Bucket and two MP3 objects, (2) bucket policy, (3) default encryption setting, and (4) browser Access Denied screenshot.